UNIVERSIDADE DE FORTALEZA
VICE REITORIA DE GRADUAÇÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

T310 - Redes Convergentes

Prof Bruno Lopes
bruno.batista@unifor.br

Chegamos ao momento de consolidar na prática tudo o que construímos ao longo das nossas aulas sobre a camada de aplicação: o modelo cliente-servidor, o papel dos sockets como interface entre a aplicação e o transporte, o formato textual do protocolo HTTP (request line, headers, linha em branco e body), os métodos GET e POST, os códigos de status, o papel do Content-Type e do Content-Length, e como o navegador transforma uma única página HTML em múltiplas requisições ao servidor.

O desafio de vocês será **desenvolver em dupla** um servidor HTTP funcional em Go, utilizando apenas sockets TCP (pacote `net`), sem o uso do pacote `net/http`. O servidor deve ser capaz de atender requisições reais de um navegador comum. O projeto conta com seis requisitos principais:

1. **Servidor TCP com concorrência:** Escutar em uma porta configurável via argumento de linha de comando, aceitar múltiplas conexões simultâneas utilizando goroutines e manter o servidor ativo continuamente.

2. **Parsing da requisição HTTP:** Ler os bytes crus recebidos no socket e extrair método, caminho, headers e, quando presente, o body da requisição, redirecionando o caminho `/` para `/index.html`.

3. **Servir arquivos estáticos:** Localizar e servir arquivos HTML, CSS e imagens a partir do diretório `./public/`, respondendo com o Content-Type correto para cada formato e transmitindo arquivos binários (imagens) byte a byte.

4. **Página de erro 404:** Retornar uma resposta HTTP com status 404 e uma página HTML amigável sempre que o caminho solicitado não corresponder a nenhum arquivo existente.

5. **Processamento de requisições POST:** Respeitar o Content-Length para ler o body corretamente, fazer o parsing dos dados codificados em `application/x-www-form-urlencoded` (tratando `+` como espaço e sequências `%XX`) e extrair os campos enviados pelo formulário.

6. **Resposta dinâmica ao POST:** Gerar em tempo de execução uma página HTML contendo os dados recebidos do formulário, respondendo com o status, os headers e o body adequados.

Regra de Ouro (Apresentação e Defesa Oral)

ATENÇÃO: O envio do código não garante aprovação. No momento da apresentação individual com o professor, cada estudante será arguido exclusivamente sobre os requisitos que assumiu, suas decisões de implementação e o funcionamento das operações de socket, parsing HTTP e construção de respostas utilizadas. A não demonstração de domínio técnico ou a incapacidade de explicar o próprio código resultará na INVALIDAÇÃO IMEDIATA dos requisitos em questão e atribuição de NOTA ZERO (0,0) nesses requisitos, independentemente de estarem em pleno funcionamento.

Dependências entre Requisitos

Os requisitos possuem dependências entre si. A dupla deve considerar esta ordem ao planejar o desenvolvimento:

- R1 não depende de nenhum outro requisito (é a base do servidor).
- R2 depende de R1 (precisa da conexão TCP para ler a requisição).
- R3 depende de R2 (precisa do path extraído para localizar o arquivo).
- R4 depende de R3 (é o caminho alternativo quando o arquivo não existe).
- R5 depende de R2 (precisa do método e do Content-Length extraídos).
- R6 depende de R5 (precisa dos campos decodificados para montar a resposta).

Resumidamente, há dois ramos de desenvolvimento que partem de R2:

- Ramo de arquivos estáticos: R1 → R2 → R3 → R4
- Ramo de formulário/POST: R1 → R2 → R5 → R6

Requisitos

R1 — Servidor TCP com suporte a conexões simultâneas

O servidor deve escutar conexões TCP em uma porta (ex: 9000). Quando um cliente (browser) conectar, o servidor deve tratar essa conexão em uma goroutine separada, permitindo que outros clientes se conectem ao mesmo tempo sem precisar esperar.

O servidor deve imprimir as seguintes mensagens no terminal:

Ao iniciar:

```
[SERVIDOR] Escutando na porta :9000
```

Quando um cliente conectar:

```
[CONEXÃO] Nova conexão de 127.0.0.1:54321
```

Quando a conexão encerrar:

```
[CONEXÃO] Encerrada 127.0.0.1:54321
```

Caso de teste: abrir 3 abas do browser acessando `http://localhost:9000` ao mesmo tempo. As 3 abas devem receber resposta e o terminal deve exibir 3 logs de nova conexão.

R2 — Parsing da requisição HTTP

Quando o browser acessa um endereço como `http://localhost:9000/about.html`, ele envia uma mensagem de texto pela conexão TCP. Essa mensagem é chamada de **requisição HTTP** e tem o seguinte formato:

```
GET /about.html HTTP/1.1\r\n
Host: localhost:9000\r\n
Accept: text/html\r\n
\r\n
```

A primeira linha é a **request line** e contém três partes separadas por espaço:

- O **método** — indica o que o cliente quer fazer. `GET` significa "quero obter este recurso". `POST` significa "estou enviando dados" (será usado no R5).
- O **path** — o caminho do recurso solicitado (ex: `/about.html`).
- A **versão do protocolo** — geralmente `HTTP/1.1`.

As linhas seguintes são **headers**, no formato `Chave: Valor`. Cada header traz uma informação adicional sobre a requisição. O header mais importante para esta atividade é o `Content-Length`, que indica o tamanho em bytes do corpo da requisição (usado no R5).

A requisição termina com uma **linha em branco** (`\r\n\r\n`), que separa os headers do corpo.

O servidor deve ler os bytes recebidos na conexão e extrair:

- O **método** (GET ou POST)
- O **path** do recurso solicitado
- O valor do header **Content-Length** (quando presente)

Se o path for `/`, o servidor deve tratar como `/index.html`.

Após extrair essas informações, imprimir no terminal:

```
[REQUEST] Método: GET | Path: /about.html | Content-Length: 0
```

Para uma requisição POST (será usada no R5):

```
[REQUEST] Método: POST | Path: /submit | Content-Length: 42
```

Caso de teste: acessar `http://localhost:9000/about.html` no browser e verificar no terminal que o log exibe método `GET`, path `/about.html` e Content-Length `0`. Depois acessar `http://localhost:9000/` e verificar que o log mostra path `/index.html`.

R3 — Servir arquivos estáticos (HTML, CSS e imagens)

Quando o método da requisição for GET, o servidor deve procurar o arquivo correspondente ao path dentro do diretório `./public/`. Por exemplo, se o path for `/style.css`, o servidor deve abrir o arquivo `./public/style.css`.

Se o arquivo existir, o servidor deve enviar uma **resposta HTTP** pela conexão TCP. A resposta tem o seguinte formato:

```
HTTP/1.1 200 OK\r\n
Content-Type: text/css\r\n
Content-Length: 512\r\n
\r\n
[conteúdo do arquivo aqui]
```

A primeira linha é a **status line**:

- `HTTP/1.1` — versão do protocolo.
- `200` — código de status indicando sucesso.
- `OK` — descrição textual do código.

Depois vêm os headers de resposta:

- `Content-Type` — indica o tipo do conteúdo para que o browser saiba como exibi-lo.
- `Content-Length` — indica o tamanho em bytes do conteúdo que será enviado.

Após os headers vem uma **linha em branco** (`\r\n\r\n`) e em seguida o **body** — o conteúdo do arquivo.

O servidor deve detectar o `Content-Type` correto pela extensão do arquivo:

Extensão	Content-Type

`.html`	`text/html`
`.css`	`text/css`
`.png`	`image/png`
`.jpg`	`image/jpeg`

Atenção sobre imagens: arquivos HTML e CSS são arquivos de texto e podem ser lidos como strings normalmente. Arquivos de imagem (`.png`, `.jpg`) são **arquivos binários** — eles contêm bytes que não representam texto legível. O servidor deve ler os bytes do arquivo de imagem e enviá-los diretamente na conexão, sem tentar convertê-los para string. O `Content-Length` deve refletir o tamanho real do arquivo em bytes.

Após enviar o arquivo, imprimir no terminal:

```
[ARQUIVO] Servindo /style.css (text/css, 512 bytes)
```

Para imagens:

```
[ARQUIVO] Servindo /logo.png (image/png, 20480 bytes)
```

Caso de teste: criar no diretório `./public/` um arquivo `index.html` que referencia um `style.css` e uma imagem. Acessar `http://localhost:9000/` no browser. A página deve aparecer com os estilos CSS aplicados e a imagem visível. O terminal deve exibir um log para cada arquivo servido (HTML, CSS e imagem) com os content-types corretos.

R4 — Página de erro 404

Quando o arquivo solicitado pelo path **não existir** no diretório `./public/`, o servidor deve responder com uma página de erro. A resposta segue o mesmo formato HTTP, mas com a status line indicando o erro:

```
HTTP/1.1 404 Not Found\r\n
Content-Type: text/html\r\n
Content-Length: 130\r\n
\r\n
<html>
<body>
    <h1>404 - Não Encontrado</h1>
    <p>O recurso solicitado não existe.</p>
</body>
</html>
```

A diferença está no código de status: `404` em vez de `200`, e `Not Found` em vez de `OK`. O body é uma página HTML simples informando que o recurso não foi encontrado.

Imprimir no terminal:

```
[404] Recurso não encontrado: /pagina-que-nao-existe.html
```

Caso de teste: acessar `http://localhost:9000/nao-existe.html` no browser. O browser deve exibir a página de erro com a mensagem "404 - Não Encontrado" e o terminal deve mostrar o log com o path solicitado.

R5 — Processamento de requisições POST

Quando o usuário preenche um formulário HTML e clica no botão de enviar, o browser envia uma requisição com método **POST**. Essa requisição é diferente do GET porque possui um **body** (corpo) contendo os dados preenchidos no formulário.

Exemplo: suponha um formulário com três campos (nome, email e mensagem). Se o usuário preencher o nome como "João Silva", o email como "joao@email.com" e a mensagem como "Olá", o browser enviará a seguinte requisição:

```
POST /submit HTTP/1.1\r\n
Host: localhost:9000\r\n
Content-Type: application/x-www-form-urlencoded\r\n
Content-Length: 47\r\n
\r\n
nome=João+Silva&email=joao%40email.com&msg=Olá
```

Observe que, após a linha em branco (`\r\n\r\n`), aparece o body com os dados do formulário. O servidor deve realizar os seguintes passos:

Passo 1 — Descobrir o tamanho do body: o header `Content-Length` informa quantos bytes tem o body. No exemplo acima, são 47 bytes. O servidor deve usar esse valor para saber exatamente quantos bytes precisa ler da conexão após a linha em branco.

Passo 2 — Ler o body: ler exatamente a quantidade de bytes indicada por `Content-Length`. Isso é o body da requisição.

Passo 3 — Separar os campos: o body está no formato `chave=valor`, onde os pares são separados pelo caractere `&`. No exemplo:

- `nome=João+Silva`
- `email=joao%40email.com`
- `msg=Olá`

Passo 4 — Decodificar os valores: o browser aplica uma codificação chamada **URL encoding** nos valores antes de enviar. As duas substituições mais comuns são:

- O caractere **espaço** é substituído por **+**. Então `João+Silva` deve ser interpretado como `João Silva`.
- Caracteres especiais são substituídos por **`%XX`**, onde XX é o código hexadecimal do caractere. Por exemplo, `%40` representa `@`. Então `joao%40email.com` deve ser interpretado como `joao@email.com`.

Dica: em Go, a função `url.QueryUnescape()` do pacote `net/url` realiza essa decodificação. O uso desse pacote é **permitido** pois ele trata de codificação de texto, não do protocolo HTTP.

Após decodificar os campos, imprimir no terminal:

```
[POST] Body bruto (47 bytes): nome=João+Silva&email=joao%40email.com&msg=Olá
[POST] Campo: nome = João Silva
[POST] Campo: email = joao@email.com
[POST] Campo: msg = Olá
```

Após processar os campos, o servidor deve enviar uma resposta ao browser — essa resposta é definida no **R6**.

Caso de teste: preencher o formulário com o nome "João Silva" e enviar. Verificar no terminal que o log do body bruto mostra `João+Silva` (com **+**) e o log do campo decodificado mostra `João Silva` (com espaço).

R6 — Resposta dinâmica ao POST

Após processar os dados do formulário (R5), o servidor deve montar uma página HTML **em tempo de execução** que exiba os dados recebidos e enviá-la como resposta ao browser. A resposta segue o formato HTTP padrão:

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 256\r\n
\r\n
<html>
<body>
    <h1>Dados Recebidos</h1>
    <ul>
        <li><strong>nome:</strong> João Silva</li>
        <li><strong>email:</strong> joao@email.com</li>
        <li><strong>msg:</strong> Olá</li>
    </ul>
</body>
</html>
```

O HTML acima **não é fixo** — ele deve ser **construído pelo código** a partir dos campos decodificados no R5. Se o formulário tiver 5 campos, a página deve listar os 5. Se tiver 3, lista 3. O servidor monta o HTML dinamicamente usando os dados recebidos.

O `Content-Length` deve corresponder ao tamanho real do HTML gerado em bytes.

Imprimir no terminal:

```
[RESPONSE] Resposta dinâmica enviada (text/html, 256 bytes)
```

Caso de teste: enviar o formulário preenchido e verificar que o browser exibe uma página listando todos os campos com seus valores corretos. Testar com valores que contenham espaços (ex: "João Silva") e acentos (ex: "Programação") e verificar que aparecem corretamente na página exibida pelo browser.

Regras de Organização da Dupla

- Cada dupla possui **6 requisitos** para implementar.
- Cada integrante deve escolher **3 requisitos** sob sua responsabilidade.
- A escolha é **livre**, mas deve respeitar as dependências entre requisitos (seção 2) para organizar a ordem de desenvolvimento.
- No dia da apresentação, cada integrante será questionado **individualmente** sobre os requisitos que assumiu. As perguntas serão feitas na ordem de dependência dos requisitos.
- O outro integrante **não pode auxiliar** nas respostas do colega. Como a avaliação segue a ordem de dependência, os requisitos anteriores já terão sido explicados — qualquer intervenção evidencia que o responsável não domina seu próprio requisito.
- **Trabalho individual por opção:** caso o aluno opte por realizar a atividade sozinho, a nota referente ao critério de Trabalho em Equipe não será atribuída.
- **Casos excepcionais** (desistência ou ausência de um integrante) serão avaliados individualmente pelo professor.

Cláusula de Validação: O domínio técnico insuficiente na arguição oral ou inconsistência grave entre o código e o declarado sobre o uso de IA resultará na anulação total da nota (nota 0,0).

⚠️ Avisos Importantes e Instruções de Submissão

Para fins de padronização, organização e cumprimento dos prazos regimentais, todos os estudantes devem seguir estritamente as diretrizes de envio descritas a seguir:

Formato e Conteúdo do Envio:

A entrega deve ser realizada exclusivamente via Ambiente Virtual de Aprendizagem (AVA), em formato de arquivo compactado .zip. O arquivo compactado deve conter obrigatoriamente: o código-fonte completo do projeto em Go (um ou mais arquivos `.go`), o diretório `./public/` com os arquivos estáticos utilizados nos testes, o documento PDF contendo os nomes dos

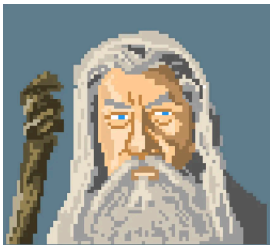
integrantes da dupla e a divisão dos requisitos (quais requisitos cada integrante assumiu) e o documento PDF da Declaração de Uso de Inteligência Artificial devidamente preenchido e assinado (via gov.br ou manuscrito escaneado). O código enviado no AVA deve ser idêntico ao apresentado no dia da defesa oral. Qualquer divergência entre o código entregue e o código demonstrado resultará na invalidação dos requisitos afetados.

Penalidades por Não Conformidade:

Ausência do PDF de divisão de requisitos: a entrega sem o documento que identifica qual integrante e responsável por quais requisitos impossibilita a avaliação individual e acarretará o desconto de 2,0 (dois) pontos na nota final da atividade. Ausência de assinatura na Declaração: a entrega da Declaração de Uso de IA sem a devida assinatura formal implicará o desconto de 5,0 (cinco) pontos na nota final do trabalho. Preenchimento vago ou inadequado da Seção 4: conforme estabelecido nas normas institucionais, a ausencia de detalhamento na descrição do uso de IA (respostas genéricas como "usei para tirar dúvidas" ou "pedir ajuda ao Chat GPT") acarretará a atribuição de nota zero (0,0) a atividade, sem possibilidade de reenvio. Uso antiético ou não autorizado de IA: o descumprimento do Guia Institucional — tal como solicitar que a ferramenta gere a solução completa, extrapolar os níveis autorizados (1 a 3) sem o log completo de cocriação, ou a incapacidade de explicar o código linha a linha na defesa oral — resultará na invalidação total do trabalho com nota zero (0,0), além de eventuais encaminhamentos disciplinares.

Canal de Submissão e Política de Atrasos:

Canal exclusivo: o AVA é o único meio aceito para envio da atividade. Trabalhos encaminhados por e-mail, redes sociais, mensagens instantâneas ou qualquer outro canal não serão recebidos nem avaliados. Submissões fora do prazo: envios posteriores a data-limite cadastrada no AVA serão aceitos, contudo, incidirá um desconto progressivo de 2,0 (dois) pontos na nota final para cada dia de atraso.



"Tudo o que temos de decidir é o que fazer com o tempo que nos é dado."
- Gandalf